Pick-and-Place Using Only Local Planners

Artificial Potential Fields & Reactive State Machines in PyBullet Simulation

Team Name: RIPP

Member 1: Sukeerthi Kattamuri (2024102075)

Member 2: Moksha Choksi (2024102048)

Member 3: Kavya Veer (2024112002)

Date: March 7, 2026

Specifications:

The object's position is known

- No pre-planned joint movements allowed
- The arm must dodge obstacles in real time

Contents

1 Problem Statement **3**

2 Tools Used **3**

3 The Core Idea **4**
3.1 The Arm Follows Virtual Forces . . . 4
3.2 A State Machine Manages the Task . . . 5
3.3 The Actual Arm Movement . . . 5
3.4 Inverse Kinematics . . . 5
3.5 Gripping Mechanism . . . 6

4 Challenges Encountered **6**
4.1 The Robot Getting Stuck in a Dead Zone . . . 6
4.2 Self-Collision . . . 6
4.3 Double Grasp . . . 6
4.4 The Arm Hovering Forever Above the Drop Zone . . . 7
4.5 The Arm Retrying the Same Blocked Path . . . 7
4.6 The Arm Missing the Block on the Way Down . . . 7
4.7 Blocks Toppling During Stacking . . . 7

5 Experiment 1: Sequential Tasks (2-Object Pickup) **7**
5.1 Implementation . . . 8
5.2 Key Code . . . 8
5.3 Result . . . 9

6 Experiment 2: Stacking Randomization (3-Object Stack) **9**
6.1 Implementation . . . 9
6.2 Key Code . . . 9
6.3 Result . . . 10

7 Experiment 3: Color Sort **10**
7.1 Implementation . . . 10
7.2 Key Code . . . 10
7.3 Also: The Hard-Clear Fix . . . 11
7.4 Result . . . 11

8 Experiment 4: 2 Objects, 1 Moving Obstacle **11**
8.1 Implementation . . . 11
8.2 Key Code . . . 12
8.3 Result . . . 12

9 Experiment 5: 1 Object, Multiple Random Obstacles **13**
9.1 Implementation . . . 13
9.2 Key Code . . . 13
9.3 Result . . . 14

10 Modular Code Structure **14**

11 Results **15**
11.1 What We Learned Across All Experiments 15

12 Media **16**
12.1 Video Link 16
12.2 Screenshots 16

13 Conclusion **17**

1 Problem Statement

The goal of this project is to make a robot arm pick up objects and place them somewhere else, but with a specific set of restrictions:

1. The object's position is known
2. No Pre-planned movements
3. Dodge obstacles in real time

We used the **Franka Panda**, a 7-joint robot arm, inside the **PyBullet** physics simulator. Five experiments were built, each one adding a new layer of difficulty.

#	Experiment Name	Blocks	Obstacles	Main challenge
1	Sequential Tasks (2-Object Pickup)	2	1 (side-to-side)	Timing around a pendulum-like obstacle
2	Stacking	3	0	Placing blocks precisely on top of each other
3	Color Sort	12	0	Sorting by color, arm self-collision
4	2 objects, 1 moving obstacle	2	1 (moving sphere)	Avoiding a roaming dynamic object
5	1 object, Multiple Random Obstacles	1	40 (mixed shapes)	Getting through a crowded field

2 Tools Used

Purpose	Tool	Explanation
Simulator	PyBullet	Runs physics, shows the 3D scene
Robot model	Franka Panda URDF	Robot Arm
Joint control	PyBullet IK solver	Converts a target position into joint angles
Obstacle logic	Custom Python (NumPy)	Calculates push/pull forces for navigation
Distance checks	PyBullet closest-point query	Measures how close the arm is to obstacles
Language	Python	Codes in Python

3 The Core Idea

Before diving into each experiment, here is the big picture of how the arm decides where to move at every moment.

3.1 The Arm Follows Virtual Forces

Think of the target location as a magnet pulling the arm toward it, and each obstacle as a magnet pushing the arm away. At every step the arm adds these two forces together and moves a tiny bit in that direction. This approach is called an **Artificial Potential Field (APF)**.

- **Pull force** - points from the arm's current position straight toward the goal. The further away the goal, the stronger the pull.
- **Push force** - points away from any nearby obstacle. The closer the obstacle, the harder the push.
- **Sidestep force** - a small sideways nudge added to the push force so the arm curves around an obstacle rather than just bouncing back.

Pull force

The pull force is simply the direction to the goal, scaled by a gain k_{att}:

$$\mathbf{f}_{att} = k_{att} \cdot (\mathbf{p}_{goal} - \mathbf{p}_{ee}) \tag{1}$$

where $\mathbf{p}_{ee}$ is the arm's current position and $\mathbf{p}_{goal}$ is where we want it to go. The further away the goal, the bigger the force, the arm naturally slows down as it gets close.

Push force

The push force points directly away from the obstacle. It grows much stronger as the arm gets close, because of the d^2 in the denominator:

$$\mathbf{f}_{rep} = \frac{k_{rep}}{d^2} \cdot \hat{\mathbf{d}}, \qquad d = \|\mathbf{p}_{ee} - \mathbf{p}_{obs}\|, \quad \text{only active when } d < 0.25\,\text{m} \tag{2}$$

where $\hat{\mathbf{d}}$ is a unit vector pointing from the obstacle toward the arm, and d is the distance between them. A sidestep component is added on top of this to break symmetry and help the arm curve sideways.

Speed clamping

After adding the two forces together, the result is clamped to a maximum speed v_{max} so the arm never moves too fast and becomes unstable:

$$\mathbf{v}_{cmd} = \begin{cases} \mathbf{f}_{att} + \mathbf{f}_{rep} & \text{if } \|\mathbf{f}_{att} + \mathbf{f}_{rep}\| \leq v_{max} \\ v_{max} \cdot \dfrac{\mathbf{f}_{att} + \mathbf{f}_{rep}}{\|\mathbf{f}_{att} + \mathbf{f}_{rep}\|} & \text{otherwise} \end{cases} \tag{3}$$

The arm then moves by exactly $\mathbf{v}_{cmd}$ each tick, and the joint angles are recalculated from scratch to match that new position.

3.2 A State Machine Manages the Task

The APF handles how to move, but we also need something to decide what to do next, for example, pick up block 1, carry it, place it, go back for block 2, and so on. This is handled by a **state machine**: a simple list of stages the robot steps through in order.

Stage	Purpose
`APPROACH`	Arm moves to hover above the block
`DESCEND`	Arm lowers straight down to grab height
`GRASP`	Block is locked to the gripper
`LIFT`	Arm raises up to a safe travel height
`GO_TO_ZONE`	Arm moves across to the drop area
`DESCEND_PLACE`	Arm lowers the block to the target spot
`RELEASE`	Block is let go
`RETREAT`	Arm lifts back up, ready for the next block
`RETURN_HOME`	Arm goes back to its starting position

3.3 The Actual Arm Movement

Once we have the desired direction of movement, we calculate the arm's next position and IK solver solves for what angle each of the 7 joint should be at to reach this position. The solver gives us those angles, and we send them to the motors. This happens 240 times per second, giving the arm smooth, continuous motion.

We keep two preferred postures for the IK solver to aim for:

- **Neutral posture** - used when the arm is moving around freely.
- **Elbow-up posture** - used when the arm is carrying a block, to keep the elbow raised and avoid hitting the floor or the base.

3.4 Inverse Kinematics

The Cartesian velocity command is integrated to get a new EE position at every step:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \dot{\mathbf{p}}_{\text{cmd}} \cdot \Delta t \tag{4}$$

PyBullet's damped-least-squares IK is then called with joint-limit and rest-pose biasing:

$$\mathbf{q}^* = \text{IK}(\mathbf{p}_{t+1},\ \mathbf{R}_{\text{home}},\ \mathbf{q}_{\text{rest}}) \tag{5}$$

Two rest-pose seeds are maintained — a neutral "unravel" pose for free-space transit and an "elbow-up" pose for carrying blocks — to reduce self-collision and singularity risk.

3.5 Gripping Mechanism

PyBullet's friction simulation is not reliable enough for finger-contact grasping at real-time speeds. So we use a two-step approach:

1. Close the fingers around the block.
2. Attach an invisible rigid link between the gripper and the block so it cannot fall.

When we want to release, the rigid link is deleted and the fingers open.

4 Challenges Encountered

Below are the biggest problems we ran into during development and the specific fixes we implemented for each one.

4.1 The Robot Getting Stuck in a Dead Zone

Problem: When the pull force toward the goal and the push forces from surrounding obstacles pointed in exactly opposite directions, they cancelled each other out. The arm had zero net force and just froze in place

Solution:

- **sidestep force** - a small sideways push that runs perpendicular to the push-away direction. This stops the arm from being perfectly balanced and encourages it to curve sideways around the obstacle instead.
- **watchdog timer**. If the arm has been in the same stage for more than 4.5 seconds without reaching its target, we assume it is stuck. The system then fires a short escape kick, a random upward movement that lifts the arm above the problem area before normal navigation resumes.

4.2 Self-Collision

Problem: The joint solver would find valid solutions that resulted in self collisions.

Solution:

- **seeded the solver** with a preferred arm shape.
- We use different seeds for different situations: a neutral posture when moving freely, and an elbow-up posture when carrying a block at low height.
- When a stall is detected, the arm also temporarily resets to its neutral posture directly before trying again. This physically untangles any awkward configuration.

4.3 Double Grasp

Problem: Occasionally the command to release the previous block would not process in time before the arm moved to the next block. The arm would then try to pick up a second block while the first was still invisibly attached to the gripper, effectively welding two blocks together.

Solution: At the very start of every APPROACH stage, we added a **hard-clear check** that forcefully deletes any existing grip constraint, no matter what. Even if nothing should be attached, we delete it anyway as a safety measure. The arm is also commanded to open its fingers wide before descending.

4.4 The Arm Hovering Forever Above the Drop Zone

Problem: The threshold for 'close enough to release' was set too tight (under 1 cm). Physics noise in the simulation meant the arm was never quite hitting that exact coordinate, so it would hover indefinitely, making tiny corrections forever.

Solutions: We relaxed the release threshold to 3 cm and added a short wait (about 60 simulation steps) after releasing. This gives the physics engine enough time to settle the block before the arm retreats.

4.5 The Arm Retrying the Same Blocked Path

Problem: After an escape kick, the arm would sometimes fly back to exactly the same position and try the exact same route again, getting stuck in the same place in a loop.

Solution: The escape kick is randomised, and the upward direction has a strong random component. This means the arm re-enters the workspace from a different angle each time, which changes the joint solution the IK solver picks and breaks the loop.

4.6 The Arm Missing the Block on the Way Down

Problem: When the arm descended to pick up a block, the obstacle-avoidance force was still active. Even a small push from a nearby object was enough to shift the gripper sideways by a centimetre or two, causing it to miss.

Solution: We turned off all obstacle avoidance during descent. The arm moves purely vertically during DESCEND and DESCEND_PLACE, with no sideways forces allowed. For experiments with an obstacle very close to the block, we pass in a fake obstacle position far away so the avoidance maths sees nothing nearby.

4.7 Blocks Toppling During Stacking

Problem: In the stacking experiment, the arm would lower the second block onto the first and the tower would immediately fall over. The issue was that the arm was making small abrupt corrections on the way down, which sent a tiny impulse through the block on contact and knocked the tower sideways.

Solution: We added velocity smoothing, instead of jumping directly to the new calculated speed, the arm blends 20% of the new speed with 80% of the last step's speed. This removes the jerkiness and makes the final placement a smooth, gentle contact. We also added extra simulation steps after each release so the block fully settles before the arm moves away.

5 Experiment 1: Sequential Tasks (2-Object Pickup)

Setup: 2 blocks, 1 large sphere that swings back and forth in a smooth sine-wave pattern across the workspace.
Goal: Transfer both blocks to their bins while navigating around the predictably moving obstacle.

5.1 Implementation

This is similar to Experiment 2 but with a predictable obstacle motion. The new thing we tried here was **distance-scaled repulsion**: as the arm gets closer to its target, the push force automatically weakens. This means near the goal the avoidance force almost disappears, giving a much smoother and more accurate final placement.

5.2 Key Code

```python
def get_reactive_vel(ee_pos, goal_pos, obs_pos):
    dist_to_goal = np.linalg.norm(np.array(goal_pos) - np.array(ee_pos))
    f_att = (np.array(goal_pos) - np.array(ee_pos)) * 3.5

    dist_to_obs = np.linalg.norm(np.array(ee_pos) - np.array(obs_pos))
    f_rep = np.zeros(3)

    if dist_to_obs < 0.25:
        f_push = ((np.array(ee_pos) - np.array(obs_pos))
                  / dist_to_obs**3) * 0.015
        f_side = np.array([-(ee_pos[1] - obs_pos[1]),
                           (ee_pos[0] - obs_pos[0]),
                           0]) * 0.02
        # Scale repulsion down as we approach the goal
        # (avoidance fades out near the destination)
        f_rep = (f_push + f_side) * dist_to_goal

    total = f_att + f_rep
    if np.linalg.norm(total) > 0.06:
        total = (total / np.linalg.norm(total)) * 0.06
    return total

# Obstacle swings left and right in a sine wave
new_obs_y = 0.1 + np.sin(time.time() * 2.0) * 0.1
p.resetBasePositionAndOrientation(
    obstacle, [0.25, new_obs_y, 0.15], [0, 0, 0, 1])

# Turn off avoidance during contact phases
if state in ["HOVER_STABILIZE", "HOVER_PICK", "DESCEND",
             "GRASP", "LIFT", "DESCEND_PLACE", "RELEASE", "RETREAT_UP"]:
    vel = get_reactive_vel(ee_pos, target, [100.0, 100.0, 100.0])
else:
    vel = get_reactive_vel(ee_pos, target, obs_pos)
```

Listing 1: Distance-scaled repulsion and sinusoidal obstacle motion

5.3 Result

Both blocks transferred cleanly. The distance-scaled repulsion made the final approach noticeably smoother compared to Experiment 2, where the arm would sometimes jitter near the bin as the avoidance force fought with the pull force at close range.

6 Experiment 2: Stacking Randomization (3-Object Stack)

Setup: 3 blocks at random positions on the table. No external obstacles.
Goal: Pick all three blocks and build a neat tower at a fixed spot, one block on top of the other.

6.1 Implementation

The arm needs to place each block precisely, even a few millimetres off and the tower leans or falls. The main problem we hit was the tower collapsing on contact. The fixes were velocity smoothing (blend old and new speed to remove jerks) and a longer settle wait after each release. We also increased the physics solver accuracy to make stacking more stable.

6.2 Key Code

```python
def reactive_velocity(ee_pos, goal, obstacles, last_vel):
    f_att = (np.array(goal) - np.array(ee_pos)) * 5.0
    f_rep = np.zeros(3)

    for obs in obstacles:
        obs_pos, _ = p.getBasePositionAndOrientation(obs)
        diff = np.array(ee_pos) - np.array(obs_pos)
        d    = np.linalg.norm(diff)
        if d < 0.25:
            f_rep += (diff / d**3) * 0.05

    raw    = f_att + f_rep
    # Blend: 20% new, 80% previous, smooths out any sudden direction changes
    smooth = 0.2 * raw + 0.8 * last_vel
    if np.linalg.norm(smooth) > 0.08:
        smooth = (smooth / np.linalg.norm(smooth)) * 0.08
    return smooth

# Stack heights for 3 layers (each block is 5 cm tall)
stack_targets = [
    [0.5, -0.2, 0.025],   # layer 1: sitting on the floor
    [0.5, -0.2, 0.075],   # layer 2: on top of layer 1
    [0.5, -0.2, 0.125],   # layer 3: on top of layer 2
]

# After releasing, wait for the physics to settle before moving on
elif state == "RELEASE":
    if grasp_constraint:
        p.removeConstraint(grasp_constraint)
        grasp_constraint = None
```

```
    for j in [9, 10]:
        p.setJointMotorControl2(robot, j, p.POSITION_CONTROL, 0.04)
    for _ in range(100):   # wait ~0.4 seconds of sim time
        p.stepSimulation()
    state = "RETREAT"
```

Listing 2: Smoothed velocity to eliminate jerks during precise placement

6.3 Result

Three-block tower built successfully every run. Velocity smoothing was the deciding fix, the tower collapsed consistently without it.

7 Experiment 3: Color Sort

Setup: 12 small cubes, 4 blue, 4 green, 4 yellow, placed randomly on the table. Three drop zones, one per color.
Goal: Pick up every block and drop it in the matching color zone, in order.

7.1 Implementation

There are no obstacles here, so the main challenge is just getting through 12 pick-and-place cycles without the arm tangling itself up. The big problem we hit was the arm folding its elbow inward when carrying blocks at low height, causing it to hit the base.

The fix was the dual-posture IK seeding described earlier: switch to the elbow-up seed whenever a block is being carried. Each drop zone also uses a small grid layout so multiple blocks of the same color do not stack on top of each other.

7.2 Key Code

```
# Neutral posture for free movement
unravel_angles  = [0, -0.5, 0, -2.0, 0, 1.5, 0.7]

# Elbow-up posture for carrying a block safely
elbow_up_angles = [0, -1.2, 0, -1.2, 0, 1.2, 0.7]

def move_robot(target_xyz, use_elbow_up=False):
    bias_pose = elbow_up_angles if use_elbow_up else unravel_angles
    joint_poses = p.calculateInverseKinematics(
        robot, ee_index, target_xyz,
        targetOrientation=home_orn,
        lowerLimits=ll, upperLimits=ul,
        jointRanges=jr, restPoses=bias_pose   # solver aims for this
            shape
    )
    for i in range(7):
        p.setJointMotorControl2(robot, i, p.POSITION_CONTROL,
                                joint_poses[i], force=500)

# Drop zone grid: blocks of the same color spread into a 2-column
    grid
```

```
drop_zone_centers = {
    "BLUE":   [0.5,  0.4, 0.02],
    "GREEN":  [0.5, -0.4, 0.02],
    "YELLOW": [-0.6, 0.0, 0.02],
}
zone_counters = {"BLUE": 0, "GREEN": 0, "YELLOW": 0}

base  = drop_zone_centers[c_color]
count = zone_counters[c_color]
dest_pos = [
    base[0] + (count // 2) * 0.08,   # shift columns
    base[1] + (count  % 2) * 0.08,   # shift rows
    base[2]
]
```

Listing 3: Two posture seeds to stop the arm folding onto itself

7.3 Also: The Hard-Clear Fix

```
if state == "APPROACH":
    # Hard-clear: destroy any leftover grip constraint from previous
        block
    if grasp_constraint is not None:
        p.removeConstraint(grasp_constraint)
        grasp_constraint = None
    # Also open fingers wide before descending
    for j in [9, 10]:
        p.setJointMotorControl2(robot, j, p.POSITION_CONTROL, 0.04)
```

Listing 4: Force-releasing any old grip at the start of every APPROACH

7.4 Result

All 12 blocks sorted correctly. The elbow-up seed completely eliminated the self-collision issue. The hard-clear fix meant the double-grasp bug never appeared across all 12 cycles.

8 Experiment 4: 2 Objects, 1 Moving Obstacle

Setup: 2 blocks, 1 sphere that wanders around the workspace on a random patrol, stopping at each waypoint for a couple of seconds (like a drone hovering).
Goal: Transfer both blocks to their bins while steering around the moving sphere.

8.1 Implementation

This was the first test of the actual obstacle avoidance. The push-away force is turned on during transit moves and turned off during descent and grasping. We also added the sidestep force here for the first time, which was critical, without it the arm would get trapped directly on the far side of the sphere from the goal.

Red lines in the simulation show the push force direction; green lines show the pull. A green trail also draws the full path the arm takes.

8.2 Key Code

```
def get_reactive_vel(ee_pos, goal_pos, obs_pos, state):
    # Pull toward goal
    f_att = (np.array(goal_pos) - np.array(ee_pos)) * 4.5

    dist_vec = np.array(ee_pos) - np.array(obs_pos)
    dist     = np.linalg.norm(dist_vec)
    f_rep    = np.zeros(3)

    # Push away only during transit (not during descend/grasp)
    if dist < 0.25 and state not in ["DESCEND", "GRASP", "DESCEND_PLACE"]:
        f_push = (dist_vec / dist**3) * 0.012      # push directly away
        f_side = np.array([-dist_vec[1],            # sidestep left/right
                           dist_vec[0],
                           0.08]) * 0.02
        f_rep = f_push + f_side

        p.addUserDebugLine(ee_pos, ee_pos + f_rep * 0.4,
                           [1, 0, 0], 3, lifeTime=0.06)  # red = push

    p.addUserDebugLine(ee_pos, ee_pos + f_att * 0.1,
                       [0, 1, 0], 2, lifeTime=0.06)      # green = pull

    total = f_att + f_rep
    mag   = np.linalg.norm(total)
    if mag > 0.07:
        total = (total / mag) * 0.07   # cap max speed
    return total

# Obstacle wanders to a new random spot every few seconds
if now - last_obs_update > hover_duration:
    obs_target_pos = [
        np.random.uniform(0.3, 0.5),
        np.random.uniform(-0.2, 0.3),
        np.random.uniform(0.15, 0.4)
    ]
    last_obs_update = now
    hover_duration  = np.random.uniform(2.0, 4.0)

curr_obs, _ = p.getBasePositionAndOrientation(obstacle)
drone_vel   = (np.array(obs_target_pos) - np.array(curr_obs)) * 0.05
p.resetBasePositionAndOrientation(
    obstacle, np.array(curr_obs) + drone_vel, [0, 0, 0, 1])
```

Listing 5: Push and sidestep forces, plus the moving obstacle logic

8.3 Result

Both blocks transferred without collision. The sidestep force was the key ingredient, removing it caused the arm to oscillate behind the drone instead of going around it.

9 Experiment 5: 1 Object, Multiple Random Obstacles

Setup: 1 block, 40 randomly placed obstacles of different shapes - boxes, spheres, and cylinders - spread across the full workspace. Pick and place positions each have a clear buffer zone of 20 cm around them.
Goal: Navigate through the crowded field to complete one pick-and-place.

9.1 Implementation

With so many obstacles of different shapes, simply measuring the distance from the arm to the obstacle's centre point is not accurate enough. A box that is 10 cm away at its nearest face looks far when you measure to its centre. So for this experiment we switched to asking PyBullet directly: "what is the actual closest surface point on this obstacle to any part of the arm?" This gives the true gap, not a rough estimate.

The watchdog escape mechanism also became essential here. With 40 obstacles, the arm regularly got trapped in local dead zones and needed the random upward kick to break free.

9.2 Key Code

```python
def get_reactive_velocity(robot_id, ee_index, target_pos,
                          obstacle_ids, escape_vec):
    ee_pos = np.array(p.getLinkState(robot_id, ee_index)[0])

    # Escape kick overrides everything when active
    if np.linalg.norm(escape_vec) > 0:
        return escape_vec

    f_att = (np.array(target_pos) - ee_pos) * 2.0
    if np.linalg.norm(f_att) > 1.0:
        f_att = (f_att / np.linalg.norm(f_att)) * 1.0

    f_rep = np.zeros(3)

    for obs in obstacle_ids:
        # Ask PyBullet for the true closest point on this obstacle
        pts = p.getClosestPoints(bodyA=robot_id, bodyB=obs,
                                 distance=0.05)
        if pts:
            valid = [pt for pt in pts if pt[3] > 0]
            if valid:
                pt   = min(valid, key=lambda x: x[8])
                dist = max(pt[8], 0.005)
                if dist < 0.05:
                    normal  = np.array(pt[7])              # push direction
                    tangent = np.cross(normal, [0, 0, 1])  # sidestep
                    mag = 0.02 * (1.0/dist - 1.0/0.05) / dist**2
                    f_rep += (normal + tangent * 0.4) * mag

    vel = f_att + f_rep
    if np.linalg.norm(vel) > 0.08:
```

```
        vel = (vel / np.linalg.norm(vel)) * 0.08
    return vel
```

Listing 6: Accurate closest-point repulsion for non-spherical obstacles

```
# If stuck in the same stage for over 4.5 seconds, fire an escape kick
if state in ["APPROACH", "GO_TO_PLACE", "RETURN_HOME"]:
    if current_time - state_start_time > 4.5:
        escape_timeout   = current_time + 1.2
        state_start_time = current_time + 1.2

        # Random direction with a strong upward bias to lift above obstacles
        dx, dy = random.uniform(-1, 1), random.uniform(-1, 1)
        dz     = random.uniform(1.0, 2.5)
        vec    = np.array([dx, dy, dz])
        escape_vector = (vec / np.linalg.norm(vec)) * 0.15

# Turn off the kick once its time is up
if current_time > escape_timeout:
    escape_vector = np.zeros(3)
```

Listing 7: Watchdog timer and random escape kick

9.3 Result

The arm completed the pick-and-place across multiple random obstacle layouts. The closest-point method noticeably reduced arm-link grazes compared to the centroid approach. The escape kick was triggered one to three times per run depending on how dense the random layout turned out.

10 Modular Code Structure

After building all five experiments as separate scripts, we refactored the shared parts into reusable files:

- `environment.py` - sets up the PyBullet world, loads the robot, spawns blocks and obstacles.
- `primitives.py` - contains reusable `pick()`, `place()`, and movement functions that any experiment can call.
- `main.py` - the actual task script, which calls the primitives.

The most important thing encoded in `primitives.py` is the separation between horizontal APF movement (where avoidance is on) and vertical movement (where avoidance is off). This is now baked into named functions rather than scattered across if-statements.

```
def move_down(robot, env, goal_z,
              tol=0.007, max_steps=400, step_size=0.004):
    """Move straight down. No obstacle forces. Used for all descents."""
    for _ in range(max_steps):
        ee = robot.get_ee_pos()
```

```
        if ee[2] - goal_z < tol:
            return True
        nz = max(goal_z, ee[2] - step_size)
        robot.move_ee([ee[0], ee[1], nz])   # only Z changes
        env.step()
    return False

def pick(robot, env, planner, block_pos, src_stack_level=0):
    bx, by = float(block_pos[0]), float(block_pos[1])
    robot.open_gripper(env, steps=50)
    move_to(robot, env, planner,
            [bx, by, hover_z(src_stack_level)])     # APF on (horizontal)
    move_down(robot, env, pre_grasp_z(src_stack_level))  # APF off (vertical)
    move_down(robot, env, grasp_z(src_stack_level),
              tol=0.005, step_size=0.002)            # APF off (final descent)
    for _ in range(30): env.step()                   # settle
    robot.close_gripper(env, steps=120)
    move_up(robot, env, hover_z(src_stack_level))    # APF off (lift)
```

Listing 8: `move_down()` - pure vertical descent with no avoidance forces

11 Results

#	Experiment	Passed?	Blocks	What made it work
1	Sequential Tasks (2-Object Pickup)	Yes	2/2	Distance-scaled repulsion
2	Stacking	Yes	3/3	Velocity smoothing
3	Color Sort	Yes	12/12	Elbow-up IK seed
4	2 objects, 1 moving obstacle	Yes	2/2	Sidestep force
5	1 object, multiple random obstacles	Yes	1/1	Closest-point repulsion + escape kick

11.1 What We Learned Across All Experiments

- **Turning off avoidance during descent is not optional.** Every single time we left it on, the arm missed the block. It has to be a clean straight-down move.
- **The sidestep force is what makes avoidance actually work.** Without it, the push force alone just pushes the arm back and it ends up in a loop.
- **Closest-point queries are worth the extra computation** when obstacles have irregular shapes. Centre-point estimates lead to phantom collisions and missed repulsions.
- **The escape kick is a last resort.** It works, but it is random. In really dense scenes it sometimes needed two or three tries before the arm found a clear path.

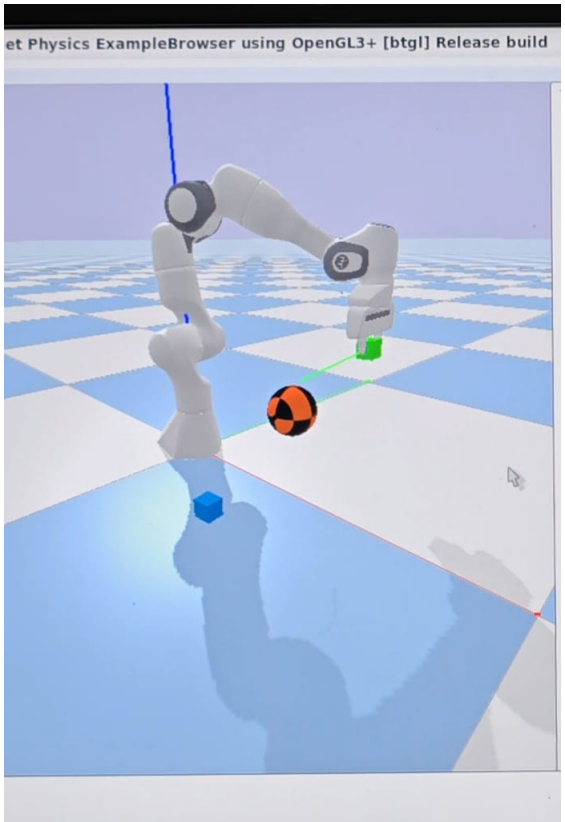
- **Smoothing matters more than you would think for stacking.** Even a tiny velocity spike on contact is enough to topple a tower.

12 Media

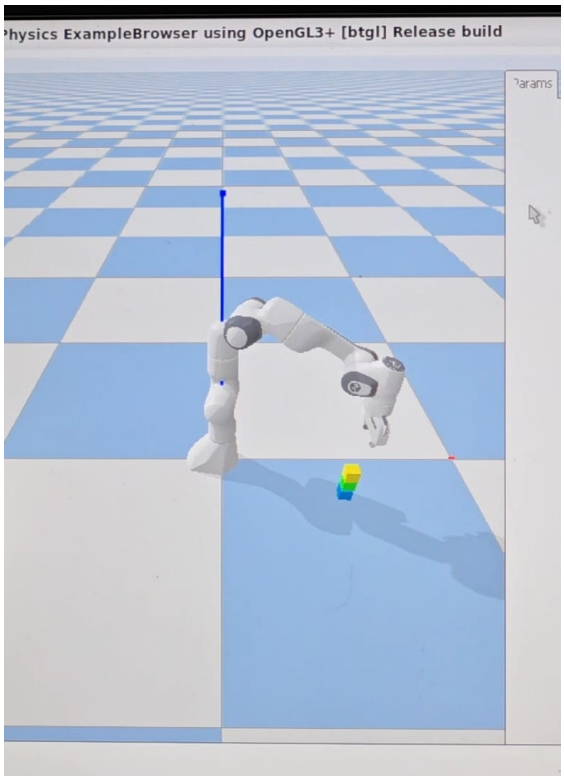
12.1 Video Link

Project Files and Demo Videos

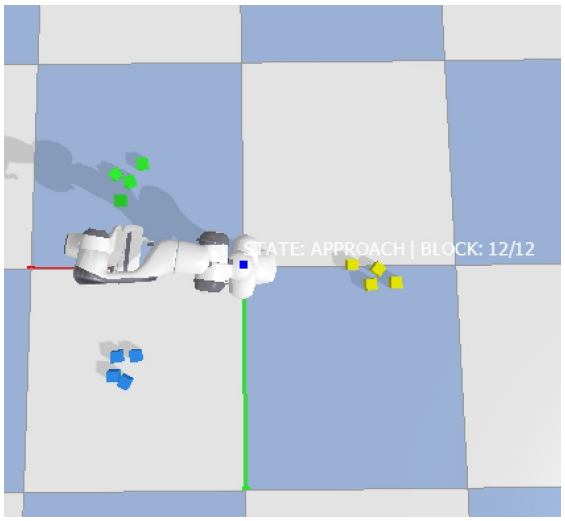
12.2 Screenshots



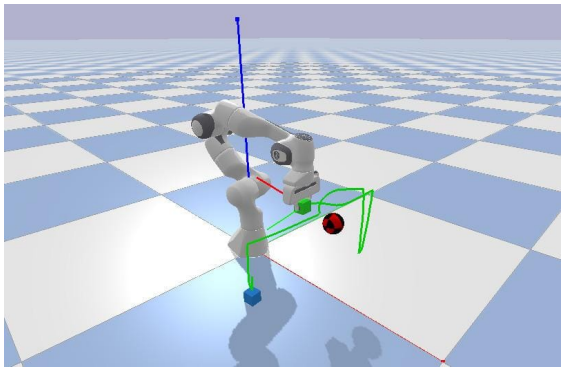
(a) Exp. 1: Arm curving around the swinging sphere.



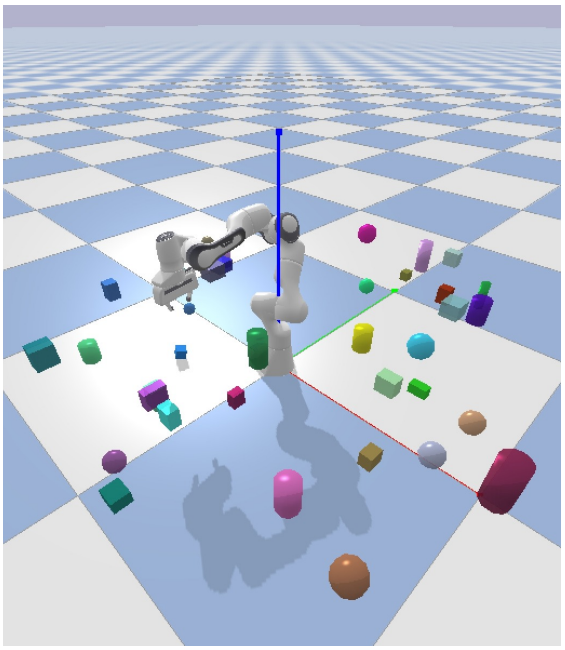
(b) Exp. 2: Three-block tower.



(a) Exp. 3: 12 blocks sorted into their color zones



(b) Exp. 4: Red push arrows and green pull arrows shown live.



(a) Exp. 5: Arm threading through the 40-obstacle field.

13 Conclusion

This project showed that a robot arm can handle quite complex pick-and-place tasks without ever pre-planning a path. By combining simple push-and-pull forces for navigation with a state machine that switches between phases, the arm could sort 12 objects, dodge a moving obstacle, navigate 40 obstacles, stack blocks precisely, and track a swinging sphere, all using only what it could sense at that exact moment.

The biggest lesson was that the state machine is just as important as the avoidance maths. Knowing *when* to apply avoidance forces (and when to switch them off) made the

difference between an arm that works reliably and one that misses the block every time. A global planner would have handled these tasks with a smoother path, but this fully reactive approach proved surprisingly capable, and every problem we hit had a concrete, implementable fix.